SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1723

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 3 **Duration : 1 Hour**
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I Daniel Jesuraji(192565085) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift

   ii. D2 must work before D3 (Morning < Afternoon < Night)

   iii. Only one doctor per shift

   iv. D3 cannot work Morning

   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

1. Introduction

Backtracking Search is an uninformed search technique that solves problems by assigning values to variables one at a time. Whenever an assignment violates a constraint, the algorithm rejects that assignment and returns to the previous step to try another value.

In this problem, three doctors, D1, D2 and D3, must be assigned to three shifts: Morning, Afternoon and Night.

2. Given Constraints

1. D1 cannot work the Night shift.
2. D2 must work before D3, according to the order Morning < Afternoon < Night.
3. Only one doctor can be assigned to each shift.
4. D3 cannot work the Morning shift.
5. Every doctor must be assigned exactly one shift.

3. Variables and Domains

The variables are D1, D2 and D3.

The domain of each variable is:

\[D=\{\text{Morning, Afternoon, Night}\} \]

The objective is to find a complete assignment that satisfies all the constraints.

4. Step-by-Step Backtracking Search

Step 1: Assign a shift to D1

First, assign D1 to the Morning shift.

Assignment: D1 = Morning.

Constraint check: D1 is not assigned to Night, so the assignment is valid.

Step 2: Assign a shift to D2

The Morning shift is already occupied by D1. Therefore, assign D2 to the Afternoon shift.

Assignment: D2 = Afternoon.

Constraint check: The shift is available, so the assignment is valid.

Step 3: Assign a shift to D3

D3 cannot work the Morning shift. Also, D2 must work before D3.

Since D2 works in the Afternoon, D3 must work in a later shift. Therefore, assign D3 to Night.

Assignment: D3 = Night.

Constraint check: D3 is not assigned to Morning, and D2 works before D3. Hence, the assignment is valid.

Step 4: Verify all constraints

Constraint	**Verification**	**Status**
D1 cannot work Night	D1 works Morning	Satisfied
D2 must work before D3	Afternoon is before Night	Satisfied
One doctor per shift	All shifts have different doctors	Satisfied
D3 cannot work Morning	D3 works Night	Satisfied
Each doctor gets one shift	All doctors have one assignment	Satisfied

5. Final Valid Schedule

Shift	**Assigned Doctor**
Morning	D1
Afternoon	D2
Night	D3

6. Conclusion

Backtracking Search successfully finds a valid schedule by assigning shifts one at a time and checking the constraints after every assignment. The final schedule satisfies all the given conditions. This method is useful for scheduling problems because it avoids continuing with assignments that violate the constraints.

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost**.**

1. Introduction

Breadth-First Search (BFS) is an uninformed search algorithm that explores nodes level by level. It uses a FIFO (First In, First Out) queue to store the nodes waiting to be expanded.

In this problem, a robot must travel from the Start (S) to the Goal (G) in a 5×5 grid while avoiding obstacles. Each movement has a cost of 1.

Note: The question paper does not specify the actual positions of the start, goal or obstacles. The following grid is an illustrative example**.**

2. Given Constraints

1. The robot can move Up, Down, Left and Right.
2. The cost of every movement is 1.
3. The robot cannot move through obstacles.
4. Manhattan Distance is used as the heuristic measure.
5. The robot must find the shortest path from S to G.

3. Manhattan Distance Heuristic

Manhattan Distance estimates the minimum number of horizontal and vertical moves between two positions.

\[h(n)=|x_n-x_G|+|y_n-y_G| \]

Where:

- \(x_n,y_n\) are the coordinates of the current node.
- \(x_G,y_G\) are the coordinates of the goal.

For example, if the start is at (0,0) and the goal is at (4,4):

\[h(S)=|0-4|+|0-4|=8 \]

Therefore, the Manhattan Distance is 8.

Important: Standard BFS does not use the heuristic to prioritize nodes. It explores nodes level by level. Manhattan Distance is shown as an estimate; prioritizing nodes using this heuristic would be part of A* Search.

5. BFS Search Algorithm

1. Insert the starting node into the queue.
2. Mark the starting node as visited.
3. Remove the front node from the queue.
4. Check whether the removed node is the goal.
5. Generate its neighbouring cells.
6. Reject cells that are outside the grid, blocked or already visited.
7. Add all valid neighbours to the queue and record their parent nodes.
8. Repeat until the goal is reached or the queue becomes empty.
9. Reconstruct the path using the recorded parent nodes.

6. Step-by-Step Node Expansion and Queue Updates

Coordinates are represented as (row, column), starting from (0,0). Neighbours are considered in the order Right, Down, Left, Up.

Step	Expanded Node	Action	Queue After Expansion
1	(0,0)	Add (0,1)	[(0,1)]
2	(0,1)	Add (0,2)	[(0,2)]
3	(0,2)	Add (1,2)	[(1,2)]
4	(1,2)	Add (2,2)	[(2,2)]
5	(2,2)	Add (2,3), (2,1)	[(2,3), (2,1)]
6	(2,3)	Add (2,4)	[(2,1), (2,4)]
7	(2,1)	Add (2,0)	[(2,4), (2,0)]
8	(2,4)	Add (1,4), (3,4)	[(2,0), (1,4), (3,4)]
9	(2,0)	Add (3,0)	[(1,4), (3,4), (3,0)]
10	(1,4)	Add (0,4)	[(3,4), (3,0), (0,4)]
11	(3,4)	Add goal (4,4)	Goal discovered

The search terminates when the goal is reached and the shortest path is reconstructed using parent pointers.

7. Optimal Path

The path obtained for this example is:

$$\begin{aligned} S &\rightarrow (0,1) \rightarrow (0,2) \\ &\rightarrow (1,2) \rightarrow (2,2) \\ &\rightarrow (2,3) \rightarrow (2,4) \\ &\rightarrow (3,4) \rightarrow G \end{aligned}$$

8. Path Cost Calculation

Each movement costs 1.

Number of moves = 8.

\[\text{Total Cost}=8\times1=\boxed{8} \]

The Manhattan Distance from S to G is also 8. Since the path achieves this lower bound without crossing an obstacle, it is optimal for the example grid.

9. Conclusion

BFS finds the shortest path in an unweighted grid because it explores nodes level by level. The queue maintains the order of exploration, while the visited list prevents repeated processing. The Manhattan Distance estimates the minimum remaining distance, although standard BFS does not use it to prioritize nodes. The exact path and cost for the assessment depend on the actual obstacle layout.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.

c) Demonstrate how the robot applies AI concepts such as:

- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

(a) Analyze the Constraints and Explain Their Effects on Decision-Making

1. Introduction

An autonomous rescue robot is deployed in a disaster-affected building to locate a survivor. The building is represented as a grid containing free cells, obstacles and risky zones.

The robot must travel from the starting position S to the goal G while minimizing the total movement cost and ensuring safe navigation. Since the robot can observe only adjacent cells, it must make decisions using limited information and update its knowledge dynamically.

2. Analysis of Constraints

1. Movement restrictions

The robot can move only Up, Down, Left and Right. It cannot move diagonally. Therefore, it must consider only the four adjacent cells when selecting its next action.

2. Obstacles

Some cells are blocked and cannot be traversed. The robot must identify known obstacles and avoid them. If a new obstacle is discovered during movement, it must update its map and find another route.

3. Limited sensing capability

The robot can observe only adjacent cells. It does not have complete knowledge of the entire building at the beginning. Therefore, it must maintain an internal map of known free cells, obstacles, risky zones and unexplored cells.

4. Movement cost

Every normal movement costs 1. Entering a risky zone adds an additional cost of 2.

Thus,

$$\text{Risky move cost}=1+2=3$$

The robot must compare the total cost of different routes rather than considering only the number of movements.

5. Safety requirement

The robot must avoid known obstacles and consider the additional cost associated with risky zones. A route with fewer steps may not always be the safest or least expensive route.

6. Dynamic knowledge

The robot must update its knowledge after every observation. If it discovers a blocked path or a risky zone, it must revise its route. This makes the problem an online-search problem.

Conclusion for Part (a)

These constraints require the robot to use local sensing, cost-based path planning and continuous map updates. The robot must select a route that reaches the survivor while respecting movement restrictions, avoiding known obstacles and minimizing accumulated cost.

(b) Develop a Solution Using Uniform Cost Search and an Online Search Agent

1. Selected Search Strategy

Uniform Cost Search (UCS) is a search algorithm that expands the node with the lowest accumulated path cost.

It is suitable for this problem because normal and risky movements have different costs. The robot can combine UCS with an online-search agent that senses its surroundings, updates its map and replans as new information becomes available.

2. Cost Function

The path cost is calculated as:

$$g(n)=\sum_{i=1}^{n}c_i$$

Where c_i is the cost of each movement.

For a normal cell:
\[c_i=1 \]
For a risky cell:
\[c_i=1+2=3 \]
The robot selects the available path with the smallest accumulated cost.

3. Steps Involved in the Solution
Step 1: Initialization
The robot identifies its starting position S and initializes its accumulated path cost to zero. It creates an internal map containing the cells it knows about.
Step 2: Sense the environment
The robot observes adjacent cells and identifies known obstacles, free cells and risky zones.
Step 3: Generate possible actions
The robot considers movement in the four permitted directions. It rejects movements that lead outside the grid or into known blocked cells.
Step 4: Calculate movement costs
The robot assigns a cost of 1 to a normal move and a cost of 3 to entering a risky zone.
Step 5: Apply Uniform Cost Search
The robot maintains a priority queue ordered by accumulated path cost. It expands the available path with the lowest cost first.
Step 6: Move to the selected cell
The robot executes the next movement along the selected route.
Step 7: Update the internal map
After moving, the robot senses adjacent cells and updates its knowledge. Newly discovered obstacles and risky zones are added to the map.
Step 8: Replan if necessary
If the current route becomes blocked or a cheaper route becomes known, the robot updates its search and selects another suitable route.
Step 9: Reach the goal
The process continues until the robot reaches G. It then calculates and reports the total accumulated cost of the route.

4. Example of Cost Comparison
Consider two possible routes.
- Route A: 6 normal movements.
\[C_A=6\times1=6 \]
- Route B: 4 normal movements and one risky movement.
\[C_B=4+(1+2)=7 \]
Therefore, Route A has a lower total cost than Route B, even though it requires more movements.
This example shows why the robot must consider the cost of risky zones when planning its route.

Conclusion for Part (b)
Combining UCS with online search allows the robot to choose low-cost routes while adapting to newly discovered information. UCS minimizes accumulated path cost on a fixed, fully represented cost map, while online sensing and replanning help the robot handle incomplete environmental knowledge.

(c) Demonstrate the Application of AI Concepts

1. Intelligent Agents

An intelligent agent is a system that perceives its environment and performs actions to achieve a goal.

In this problem, the rescue robot acts as an intelligent agent.

- Sensors: Observe adjacent cells.
- Internal state: Stores the known map and current position.
- Decision-making: Selects a suitable movement using search algorithms.
- Actuators: Execute the selected movement.
- Goal: Reach the survivor while minimizing the total cost and avoiding known obstacles.

The robot repeatedly follows the perception–decision–action cycle.

2. Rationality

Rationality means selecting the action expected to produce the best result based on the information available to the agent.

In this problem, a rational robot:

1. Avoids known blocked cells.
2. Considers the additional cost of risky zones.
3. Selects the lowest-cost known route.
4. Updates its decisions when new information becomes available.
5. Replans when its current route is no longer suitable.

A rational robot does not need complete knowledge of the building. It must make the best decision possible using its current information.

3. Environment Type

The rescue robot operates in the following environment types:

Partially observable: The robot can observe only adjacent cells and cannot initially see the entire building.

Sequential: Each movement affects the robot's current position and its future movement choices.

Deterministic under the stated movement model: A legal movement has a defined result, assuming no additional uncertainty is introduced.

Discrete: The building is represented as a grid of individual cells, and the robot uses four discrete movement actions.

Single-agent: Only one robot is described as making decisions in the environment.

Initially unknown: The robot discovers obstacles and risky zones while exploring. Its knowledge changes as it receives new observations.

Conclusion for Part (c)

The rescue robot demonstrates intelligent-agent behaviour by sensing its surroundings, selecting actions, updating its internal state and pursuing a goal. Rationality guides its decisions, while the partially observable and initially unknown environment makes continuous observation and replanning necessary.

(d) Justify Why the Chosen Approach Is Suitable

Uniform Cost Search combined with an online-search agent is suitable for the rescue robot because the problem includes different movement costs and incomplete environmental knowledge.

1. Handles different movement costs

UCS considers the accumulated cost of a route. It can account for the extra cost of entering risky zones instead of selecting a route based only on distance.
2. Finds the least-cost path
For a fixed, fully represented map with non-negative costs, UCS finds a least-cost path to the goal. It expands paths in increasing order of accumulated cost.
3. Supports changing knowledge
The robot has limited sensing capability and discovers obstacles while moving. An online-search agent can update its internal map and replan when new information becomes available.
4. Helps avoid known obstacles
The robot can exclude known blocked cells from its legal actions. This helps it navigate safely within the limits of its available information.
5. Supports rational decision-making
By comparing accumulated costs and updating its plan based on observations, the robot can choose a suitable route toward the survivor.
6. More suitable than standard BFS
BFS is suitable for finding the shortest path when every movement has the same cost. However, in this problem, entering a risky zone adds an extra cost. Therefore, UCS is more suitable for minimizing the total path cost.

Limitation
UCS is optimal for a fixed cost map, but the robot initially has only partial information. Therefore, its route is optimal relative to the map and costs available to the search at that time. Newly discovered obstacles or risky zones may require replanning. Since the question does not provide a specific building grid, an exact numerical route and total cost cannot be calculated.

Final Conclusion
The autonomous rescue robot can solve the problem by combining intelligent-agent concepts, rational decision-making, Uniform Cost Search and online replanning. This approach enables it to consider movement costs, avoid known obstacles, update its environmental knowledge and select a low-cost route toward the survivor.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and	Provides reasonable analysis with	Limited analysis; weak prioritization	No meaningful analysis or prioritization

		prioritizes them logically	some prioritization		
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided